Отчёт о выполнении лабораторной работы по разработке по для вычисления коэффициентов поглощения $\alpha(S)$ и $\xi(S)$ n-мерного симплекса

Преподаватель: *Невский М.В.*

Выполнили: *Слелин А.В., Гаджигасанов Т.З*

8 апреля 2026 г.

Содержание

1 Введение **3**

2 Теоретические основы **3**
2.1 Базисные многочлены Лагранжа 3
2.2 Осевые диаметры 3
2.3 Коэффициент $\alpha(S)$ 4
2.4 Коэффициент $\xi(S)$ 4

3 Описание программной реализации **4**
3.1 Функции модуля simplex.py 4
3.2 Визуализация 5

4 Исходный код программы **5**
4.1 main.py 5
4.2 simplex.py 7
4.3 visualization.py 9

5 Визуализация **13**
5.1 Примеры для $\mathbb{R}^1$ 13
5.2 Примеры для $\mathbb{R}^2$ 16
5.3 Примеры для $\mathbb{R}^3$ 22
5.4 Примеры для $\mathbb{R}^5$ 28

6 Заключение **28**

1 Введение

В учебном пособии М.В. Невского и А.Ю. Ухалова «Избранные задачи анализа и вычислительной геометрии. Часть I» (ЯрГУ, 2020) рассматриваются экстремальные задачи, связанные с числовыми характеристиками конечномерных выпуклых тел, в первую очередь симплексов и кубов. Одними из центральных величин являются коэффициенты поглощения $\alpha(S)$ и $\xi(S)$, определяемые для симплекса $S \subset \mathbb{R}^n$ относительно единичного куба $Q_n = [0,1]^n$:

$$\alpha(S) = \min\{\sigma > 0 : Q_n \subset \sigma S + t \text{ для некоторого } t \in \mathbb{R}^n\}, \qquad \xi(S) = \min\{\sigma \geq 1 : Q_n \subset \sigma S\}.$$

Эти величины имеют важные приложения в вычислительной геометрии и теории интерполяции функций многих переменных.

Целью данной работы являлась разработка программы на языке Python, которая:

- по заданному набору вершин невырожденного n-мерного симплекса S вычисляет $\alpha(S)$ и $\xi(S)$;
- определяет осевые диаметры $d_i(S)$ и концы единственных отрезков максимальной длины, параллельных координатным осям;
- визуализирует симплекс, единичный куб, осевые диаметры и гомотетичные образы αS и ξS для размерностей $n = 1, 2, 3$.

Программа реализована с использованием библиотек NumPy и Matplotlib, строго опирается на формулы и теоремы из указанного пособия.

2 Теоретические основы

2.1 Базисные многочлены Лагранжа

Для невырожденного симплекса S с вершинами $x^{(j)} = (x_1^{(j)}, \ldots, x_n^{(j)})$, $j = 1, \ldots, n+1$, строится матрица

$$\mathbf{A} = \begin{pmatrix} x_1^{(1)} & \ldots & x_n^{(1)} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ x_1^{(n+1)} & \ldots & x_n^{(n+1)} & 1 \end{pmatrix},$$

определитель которой $\Delta = \det \mathbf{A}$ связан с объёмом: $\mathrm{vol}(S) = |\Delta|/n!$. Базисные многочлены Лагранжа $\lambda_j(x) = l_{1j}x_1 + \cdots + l_{nj}x_n + l_{n+1,j}$ определяются как элементы обратной матрицы $\mathbf{A}^{-1} = (l_{ij})$; они обладают свойствами

$$\lambda_j(x^{(k)}) = \delta_{jk}, \qquad \sum_{j=1}^{n+1} \lambda_j(x) = 1, \qquad \sum_{j=1}^{n+1} \lambda_j(x) x^{(j)} = x.$$

Эти соотношения подробно описаны в §1.2 пособия (стр. 7–8).

2.2 Осевые диаметры

Для каждого направления координатной оси x_i в симплексе существует единственный отрезок максимальной длины $d_i(S)$, параллельный этой оси. Согласно теореме 1.1 (стр. 10) его длина выражается через элементы обратной матрицы:

$$\frac{1}{d_i(S)} = \frac{1}{2} \sum_{j=1}^{n+1} |l_{ij}|, \qquad i = 1, \ldots, n.$$

Концы этого отрезка вычисляются по формулам (1.13)–(1.14) (стр. 12):

$$y_-^{(i)} = \sum_{j=1}^{n+1} s_{ij} x^{(j)}, \quad y_+^{(i)} = \sum_{j=1}^{n+1} t_{ij} x^{(j)}, \quad s_{ij} = \frac{|l_{ij}| - l_{ij}}{\sum_k |l_{ik}|},\ t_{ij} = \frac{|l_{ij}| + l_{ij}}{\sum_k |l_{ik}|}.$$

2.3 Коэффициент $\alpha(S)$

Из теоремы 1.3 (стр. 20) для $C = Q_n$ получаем

$$\alpha(S) = \sum_{i=1}^{n} \frac{1}{d_i(S)} = \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n+1} |l_{ij}|. \tag{1.36}$$

Эта формула удобна для вычислений, так как требует лишь элементов $\mathbf{A}^{-1}$.

2.4 Коэффициент $\xi(S)$

По определению $\xi(S) = \min\{\sigma \geq 1 : Q_n \subset \sigma S\}$. Из теоремы 1.2 (стр. 16) и леммы 1.4 (стр. 20) следует формула

$$\xi(S) = (n+1) \max_{1 \leq j \leq n+1} \max_{x \in \mathrm{ver}(Q_n)} (-\lambda_j(x)) + 1. \tag{1.25}$$

Для каждого j максимум по вершинам куба достигается в одной из вершин и может быть найден аналитически:

$$\max_{x \in \mathrm{ver}(Q_n)} (-\lambda_j(x)) = \sum_{i:\, l_{ij} < 0} |l_{ij}| - l_{n+1,j}.$$

Если полученное значение $\xi(S)$ оказывается меньше 1 (что означало бы $Q_n \subset S$), следует вернуть 1.

3 Описание программной реализации

Программа состоит из нескольких модулей:

simplex.py – основные функции для чтения данных, построения матрицы $\mathbf{A}$, вычисления обратной матрицы, осевых диаметров, $\alpha(S)$ и $\xi(S)$.

visualization.py – функции визуализации для размерностей $n = 1, 2, 3$ (отрезок, треугольник, тетраэдр) с отображением единичного куба, симплекса, осевых диаметров и гомотетичных образов αS и ξS.

main.py – точка входа: обрабатывает аргументы командной строки (файл с симплексом), выполняет вычисления и вызывает визуализацию, если размерность поддерживается.

3.1 Функции модуля simplex.py

- `read_simplex(filename)` – читает файл, содержащий в первой строке размерность n, затем $n+1$ строк с координатами вершин. Возвращает n и массив вершин.
- `build_A(vertices)` – строит матрицу $\mathbf{A}$ размера $(n+1) \times (n+1)$ путём добавления столбца единиц к координатам вершин.

- `compute_inverse(A)` – вычисляет $\mathbf{A}^{-1}$ с помощью `numpy.linalg.inv`.
- `compute_alpha(A_inv, n)` – вычисляет $\alpha(S)$ по формуле (1.36) как полусумму модулей элементов первых n строк $\mathbf{A}^{-1}$.
- `compute_xi(A_inv, n)` – вычисляет $\xi(S)$ согласно (1.25) с аналитическим определением максимума для каждого j; возвращает максимум из полученного значения и 1.
- `compute_axial_ends(A_inv, vertices)` – возвращает список пар концов осевых диаметров для каждой координатной оси, используя (1.13)–(1.14).
- `compute_center(vertices)` – центр тяжести симплекса (среднее арифметическое вершин).
- `compute_alpha_center(A_inv, vertices, alpha)` – по формуле (1.44) (стр. 26) вычисляет центр гомотетии x_0, такой что $Q_n \subset \alpha S + x_0$. Используется для визуализации αS.

3.2 Визуализация

Модуль `visualization.py` содержит функции:

- `plot_1d(vertices, axial_ends, alpha, xi, alpha_center, C_vertices=None)` – для $n = 1$ рисует отрезок $[0, 1]$, симплекс S, осевой диаметр (совпадает с S), ξS (зелёный пунктир) и αS (оранжевый пунктир) с центром в `alpha_center`.
- `plot_2d(vertices, axial_ends, alpha, xi, alpha_center, C_vertices=None)` – для $n = 2$ рисует квадрат $[0, 1]^2$, треугольник S, осевые диаметры, ξS и при аналогичных условиях αS.
- `plot_3d(vertices, axial_ends, alpha, xi, alpha_center)` – для $n = 3$ рисует каркас куба $[0, 1]^3$, тетраэдр S (полупрозрачные грани), осевые диаметры, контуры ξS (зелёный) и αS (оранжевый пунктир).

Все функции используют библиотеку Matplotlib и корректно отображают легенду, заголовок с вычисленными значениями α и ξ.

4 Исходный код программы

4.1 main.py

Листинг 1: main.py

```
import sys
import time

from colorama import init, Fore, Style
from simplex import read_simplex, build_A, compute_inverse,
    compute_alpha, compute_xi, compute_axial_ends, \
     compute_center, compute_alpha_center
from visualization import plot_1d, plot_2d, plot_3d


```

```python
def printCol(text, color):
    print(color + text + Style.RESET_ALL)


def error(text):
    printCol(text, Fore.RED)


def delimiter():
    printCol("*" * 100, Fore.YELLOW)


def main():
    if len(sys.argv) != 2:
        error("Использование: python main.py <файл_с_координатами>")
        sys.exit(1)

    init()

    start = time.perf_counter()

    filename = sys.argv[1]
    try:
        n, vertices = read_simplex(filename)
    except Exception as e:
        error(f"Ошибка чтения файла: {e}")
        sys.exit(1)

    delimiter()
    printCol(f"Размерность n = {n}", Fore.BLUE)
    printCol("Вершины:", Fore.CYAN)
    for i, v in enumerate(vertices):
        printCol(f"  x{i + 1} = ({', '.join(str(int(x) if x.is_integer() else x) for x in v)})", Fore.LIGHTCYAN_EX)
    delimiter()

    printCol("Строим матрицу A", Fore.BLUE)
    A = build_A(vertices)
    printCol("A:", Fore.CYAN)
    for row in A:
        row_str = "  " + " ".join(f"{x:10.3f}" for x in row)
        printCol(row_str, Fore.LIGHTCYAN_EX)
    delimiter()

    printCol("Ищем обратную матрицу для A", Fore.BLUE)
    A_inv = compute_inverse(A)
    printCol("  A    :", Fore.CYAN)
    for row in A_inv:
        row_str = "  " + " ".join(f"{x:10.3f}" for x in row)
        printCol(row_str, Fore.LIGHTCYAN_EX)
    delimiter()

    printCol("Вычисляем   (S) и   (S)", Fore.BLUE)
```

```
    alpha = compute_alpha(A_inv, n)
    xi = compute_xi(A_inv, n)
    printCol(f"  (S) = {alpha}", Fore.GREEN)
    printCol(f"  (S) = {xi}", Fore.GREEN)
    delimiter()

    printCol("Находим осевые диаметры и центр тяжести", Fore.BLUE)
    axial_ends = compute_axial_ends(A_inv, vertices)
    center = compute_center(vertices)
    alpha_center = compute_alpha_center(A_inv, vertices, alpha)
    printCol(f"Центр тяжести: {center}", Fore.CYAN)
    delimiter()

    end = time.perf_counter()
    printCol(f"      Программа отработала за {end - start:.6f} с", Fore
        .GREEN)
    delimiter()

    if n > 3:
        printCol("     Не могу визуализировать для размерности n > 3.",
            Fore.GREEN)
        sys.exit(0)

    printCol("     Визуализация доступна в новом окне.", Fore.GREEN)

    if n == 1:
        plot_1d(vertices, axial_ends, alpha, xi, alpha_center)
    elif n == 2:
        plot_2d(vertices, axial_ends, alpha, xi, alpha_center)
    elif n == 3:
        plot_3d(vertices, axial_ends, alpha, xi, alpha_center)

if __name__ == "__main__":
    main()
```

4.2 simplex.py

Листинг 2: simplex.py

```
import numpy as np


def read_simplex(filename):
    """
    Читает файл с координатами вершин симплекса.
    Формат: первая строка содержит целое число n (размерность),
    затем n+1 строк, каждая из которых содержит n чисел (координаты вер
        шины).
    Возвращает n и массив вершин (n+1, n).
    """
    with open(filename, 'r') as f:
        n = int(f.readline().strip())
        vertices = []
```

```python
        for _ in range(n + 1):
            line = f.readline()
            if not line:
                raise ValueError("Недостаточно строк с вершинами")
            coords = list(map(float, line.split()))
            if len(coords) != n:
                raise ValueError(f"Ожидалось {n} координат, получено {
                    len(coords)}")
            vertices.append(coords)
    return n, np.array(vertices, dtype=float)


def build_A(vertices):
    """
    Строит матрицу A размера (n+1) x (n+1):
    первые n столбцов - координаты вершин, последний столбец - единицы.
    """
    A = np.hstack([vertices, np.ones((vertices.shape[0], 1))])
    return A


def compute_inverse(A):
    """Возвращает обратную матрицу A^{-1}."""
    return np.linalg.inv(A)


def compute_alpha(A_inv, n):
    """
    Вычисляет   (S) = 0.5 * sum_{i=1..n} sum_{j=1..n+1} |l_{ij}|
    где l_{ij} - элементы A_inv (строки i, столбцы j).
    """
    # Берём только первые n строк (соответствуют координатам)
    rows = A_inv[:n, :]
    return 0.5 * np.sum(np.abs(rows))


def compute_xi(A_inv, n):
    """
    Вычисляет   (S) = (n+1) * max_j M_j + 1,
    где M_j = sum_{i: l_{ij}<0} |l_{ij}| - l_{n+1,j}.
    """
    M = []
    for j in range(n + 1):
        col = A_inv[:n, j]  # коэффициенты при переменных
        free = A_inv[n, j]  # свободный член
        neg_sum = -np.sum(col[col < 0])  # сумма модулей отрицательных
        M_j = neg_sum - free
        M.append(M_j)
    return max(1.0, (n + 1) * max(M) + 1)


def compute_axial_ends(A_inv, vertices):
    """
```

```python
    Для каждой координатной оси i (1..n) возвращает пару точек (концы отрезка)
    максимальной длины, параллельного этой оси.
    Использует формулы (1.13)-(1.14) из пособия.
    """
    n = vertices.shape[1]
    ends = []
    for i in range(n):
        row = A_inv[i, :]  # l_{i1} ... l_{i,n+1}
        s = np.abs(row) - row
        t = np.abs(row) + row
        sum_abs = np.sum(np.abs(row))
        if sum_abs == 0:
            raise ValueError(f"Нулевая строка {i + 1} в обратной матрице")
        alpha = s / sum_abs
        beta = t / sum_abs
        y_minus = np.sum(alpha[:, np.newaxis] * vertices, axis=0)
        y_plus = np.sum(beta[:, np.newaxis] * vertices, axis=0)
        ends.append((y_minus, y_plus))
    return ends


def compute_center(vertices):
    """Центр тяжести симплекса (среднее арифметическое вершин)."""
    return np.mean(vertices, axis=0)


def compute_alpha_center(A_inv, vertices, alpha):
    """
    Вычисляет центр гомотетии x0 для   (S) по формуле (1.44).
    A_inv - обратная матрица A^{-1} размера (n+1)x(n+1),
    vertices - массив вершин (n+1, n),
    alpha - значение   (S).
    Возвращает массив длины n.
    """
    n = vertices.shape[1]
    # суммы модулей элементов в столбцах (по первым n строкам)
    s_j = np.sum(np.abs(A_inv[:n, :]), axis=0)  # вектор длины n+1
    sum_s = np.sum(s_j)  # = 2*alpha
    denominator = sum_s - 2  # = 2*alpha - 2
    if abs(denominator) < 1e-12:
        raise ValueError("Знаменатель близок к нулю:   (S)     1?")
    numerator = np.zeros(n)
    for k in range(n):
        #  _j  s_j * x_k^{(j)} - 1
        numerator[k] = np.sum(s_j * vertices[:, k]) - 1
    x0 = numerator / denominator
    return x0
```

4.3 visualization.py

Листинг 3: visualization.py

```python
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np


def plot_1d(vertices, axial_ends, alpha, xi, alpha_center):
    """1D график с S, осевым диаметром,   (S)S и   (S)S."""
    fig, ax = plt.subplots(figsize=(6, 2))
    ax.grid(True)

    # Границы куба Q1 = [0,1]
    ax.axvline(0, color='k', linestyle='--', alpha=0.5)
    ax.axvline(1, color='k', linestyle='--', alpha=0.5)

    # Отрезок S
    x1, x2 = vertices.flatten()
    ax.plot([x1, x2], [0, 0], 'b-', linewidth=4, label='S')
    ax.plot(x1, 0, 'bo')
    ax.plot(x2, 0, 'bo')

    # Осевой диаметр (совпадает с S, чуть выше)
    y_minus, y_plus = axial_ends[0]
    ax.plot([y_minus[0], y_plus[0]], [0.1, 0.1], 'r-', linewidth=2, label='Осевой диаметр')

    # Гомотетичный образ   (S)S
    center = np.mean(vertices, axis=0)
    scaled_alpha = alpha_center + alpha * (vertices - alpha_center)
    xa1, xa2 = scaled_alpha.flatten()
    ax.plot([xa1, xa2], [-0.2, -0.2], 'orange', linestyle=':', linewidth=2, label='  (S)S')
    ax.plot(xa1, -0.2, 'o', color='orange')
    ax.plot(xa2, -0.2, 'o', color='orange')

    # Гомотетичный образ   (S)S
    scaled_xi = center + xi * (vertices - center)
    xs1, xs2 = scaled_xi.flatten()
    ax.plot([xs1, xs2], [-0.1, -0.1], 'g--', linewidth=2, label='  (S)S')
    ax.plot(xs1, -0.1, 'go')
    ax.plot(xs2, -0.1, 'go')

    ax.set_yticks([])
    ax.set_xlabel('x')
    ax.set_title(f'  (S) = {alpha:.6f},   (S) = {xi:.6f}', fontsize=20)
    ax.legend()
    plt.show()


def plot_2d(vertices, axial_ends, alpha, xi, alpha_center):
    """2D график с  Q  , S, осевыми диаметрами,   (S)S и   (S)S."""
    fig, ax = plt.subplots()
```

```python
    ax.grid(True)

    # Квадрат
    square = np.array([[0, 0], [1, 0], [1, 1], [0, 1], [0, 0]])
    ax.plot(square[:, 0], square[:, 1], 'k-', lw=1, label=' Q  ')

    # Симплекс (треугольник)
    tri = np.vstack([vertices, vertices[0]])
    ax.plot(tri[:, 0], tri[:, 1], 'b-', lw=2, label='S')

    # Осевые диаметры
    for i, (ym, yp) in enumerate(axial_ends):
        ax.plot([ym[0], yp[0]], [ym[1], yp[1]], 'r-', lw=2, label='Осевые диаметры' if i == 0 else '')

    # Гомотетичный образ  (S)S
    center = np.mean(vertices, axis=0)
    scaled_alpha = alpha_center + alpha * (vertices - alpha_center)
    scaled_alpha = np.vstack([scaled_alpha, scaled_alpha[0]])
    ax.plot(scaled_alpha[:, 0], scaled_alpha[:, 1], 'orange', linestyle=':', lw=2, label=f'  (S)S')

    # Гомотетичный образ  (S)S
    scaled_xi = center + xi * (vertices - center)
    scaled_xi = np.vstack([scaled_xi, scaled_xi[0]])
    ax.plot(scaled_xi[:, 0], scaled_xi[:, 1], 'g--', lw=2, label='  (S)S')

    ax.set_aspect('equal')
    ax.set_title(f'  (S) = {alpha:.6f},  (S) = {xi:.6f}', fontsize=20)
    ax.legend()
    plt.show()


def plot_3d(vertices, axial_ends, alpha, xi, alpha_center):
    """3D график с кубом  Q  , тетраэдром, осевыми диаметрами,  (S)S и (S)S."""
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')

    # Вершины куба [0,1]^3
    cube_vertices = np.array([[0, 0, 0], [1, 0, 0], [1, 1, 0], [0, 1, 0],
                              [0, 0, 1], [1, 0, 1], [1, 1, 1], [0, 1, 1]])
    # Рёбра куба
    edges = [(0, 1), (1, 2), (2, 3), (3, 0),  # нижняя грань
             (4, 5), (5, 6), (6, 7), (7, 4),  # верхняя грань
             (0, 4), (1, 5), (2, 6), (3, 7)]  # вертикальные

    for e in edges:
        ax.plot3D(*zip(cube_vertices[e[0]], cube_vertices[e[1]]), color='k', linewidth=0.5,
```

```
                    label=' Q  ' if e == (0, 1) else '')

    # Тетраэдр (грани)
    faces = [
        [vertices[0], vertices[1], vertices[2]],
        [vertices[0], vertices[1], vertices[3]],
        [vertices[0], vertices[2], vertices[3]],
        [vertices[1], vertices[2], vertices[3]]
    ]
    ax.add_collection3d(Poly3DCollection(faces, alpha=0.2, facecolor='cyan', linewidths=1, edgecolor='b', label='S'))

    # Осевые диаметры
    for i, (ym, yp) in enumerate(axial_ends):
        ax.plot3D(*zip(ym, yp), color='r', linewidth=2, label='Осевые диаметры' if i == 0 else '')

    # Гомотетичный образ  (S)S (контур)
    center = np.mean(vertices, axis=0)
    scaled_alpha = alpha_center + alpha * (vertices - alpha_center)
    alpha_edges = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
    tmp = 0
    for e in alpha_edges:
        ax.plot3D(*zip(scaled_alpha[e[0]], scaled_alpha[e[1]]), color='orange', linestyle=':', linewidth=2,
                    label='  (S)S' if tmp == 0 else '')
        tmp += 1

    # Гомотетичный образ  (S)S (контур)
    scaled_xi = center + xi * (vertices - center)
    tmp = 0
    for e in alpha_edges:
        ax.plot3D(*zip(scaled_xi[e[0]], scaled_xi[e[1]]), color='g', linestyle='--', linewidth=2,
                    label='  (S)S' if tmp == 0 else '')
        tmp += 1

    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_title(f'  (S) = {alpha:.6f},  (S) = {xi:.6f}', fontsize=20)
    ax.legend()
    plt.show()
```

5 Визуализация

5.1 Примеры для $\mathbb{R}^1$

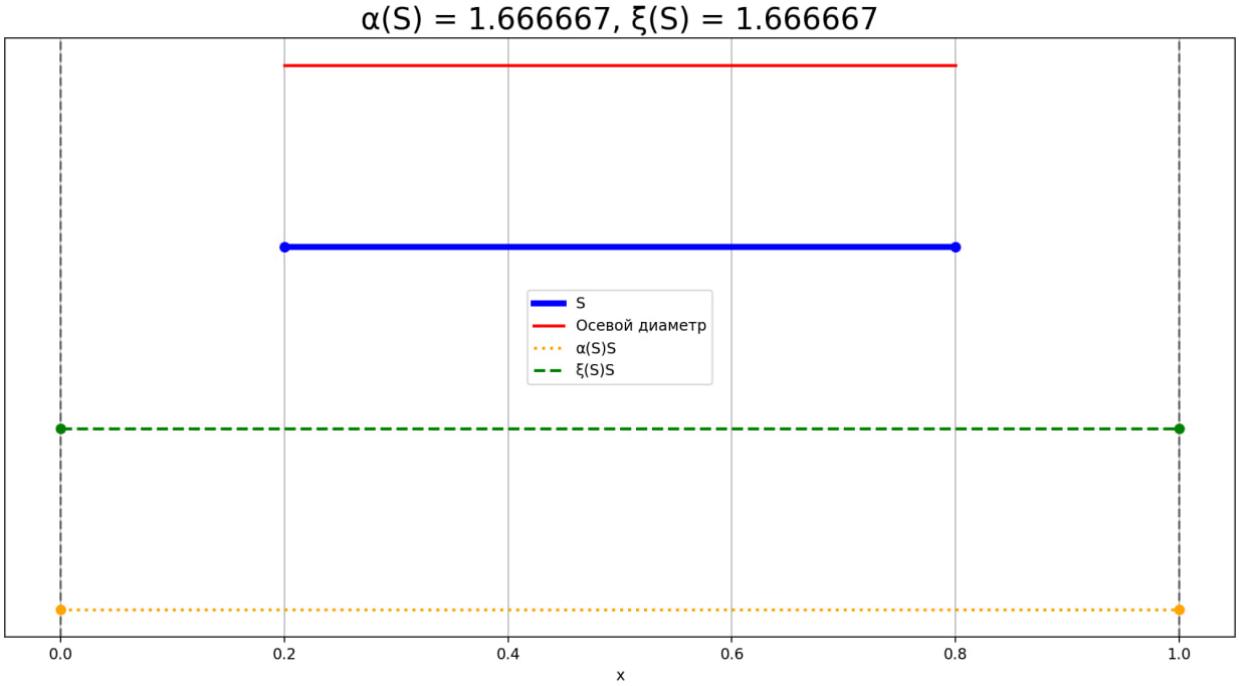

Рис. 1: S ($[0.2, 0.8]$) внутри Q ($[0, 1]$)

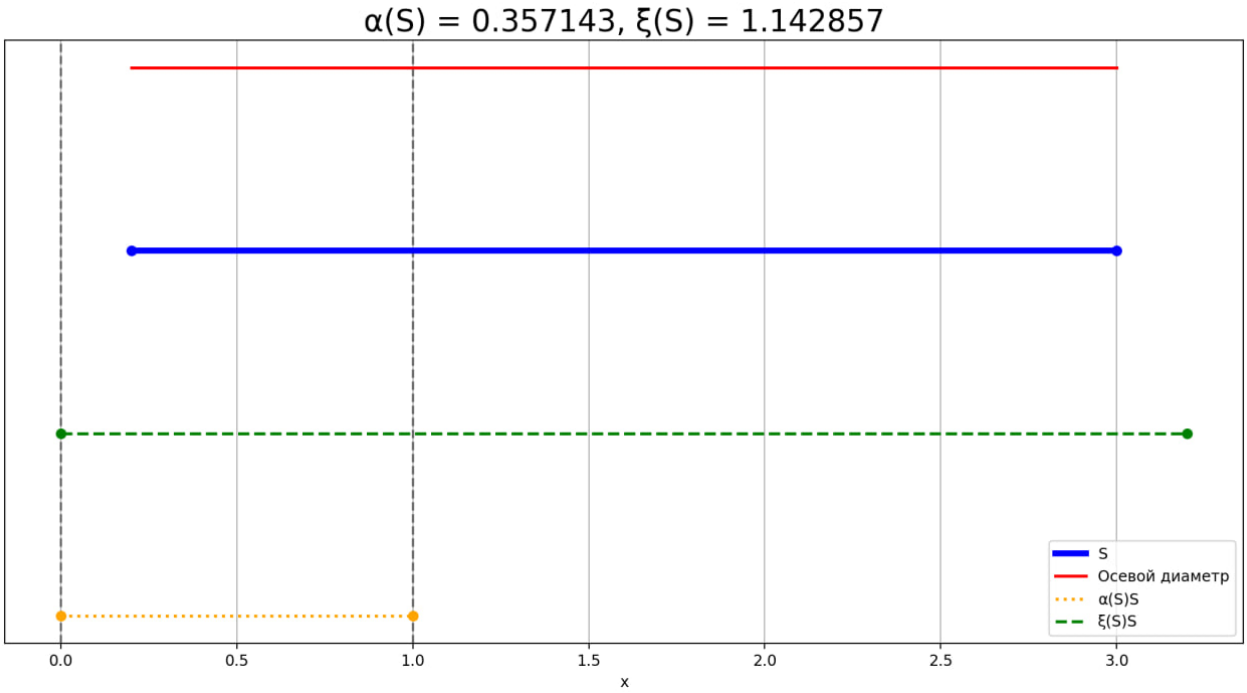

Рис. 2: S ($[0.2, 3]$) частично в Q_1 ($[0, 1]$)

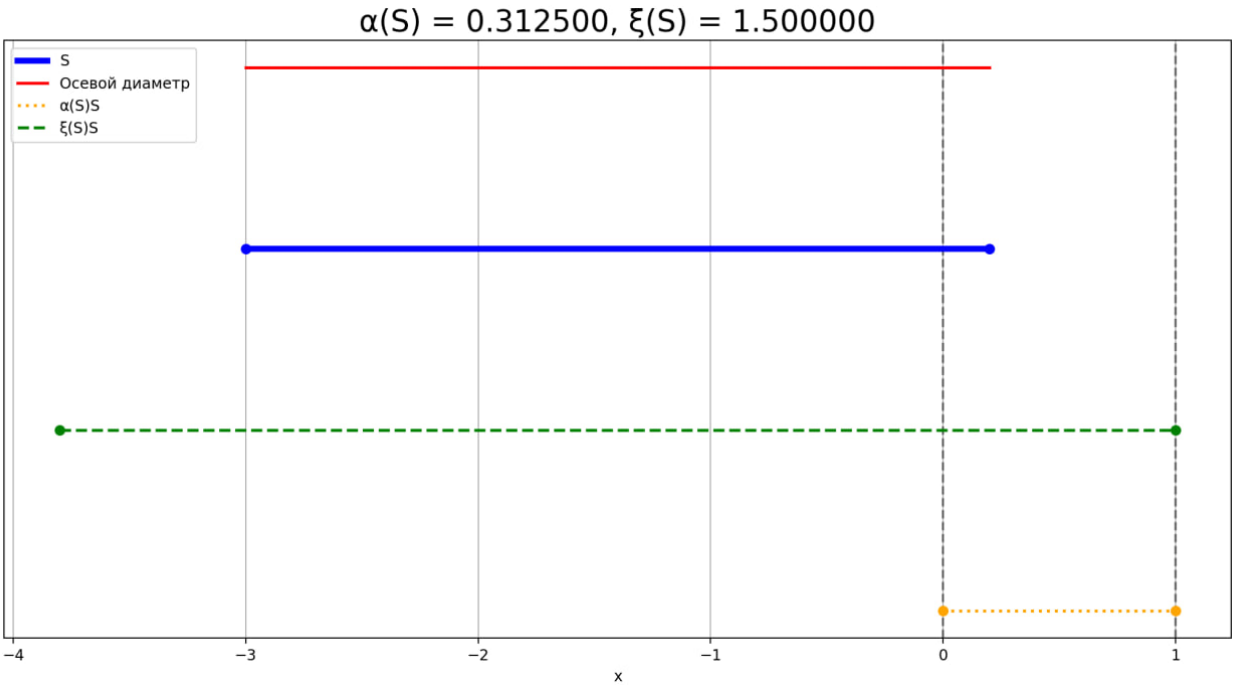

Рис. 3: S $([-3, 0.2])$ частично в Q_1 $([0, 1])$

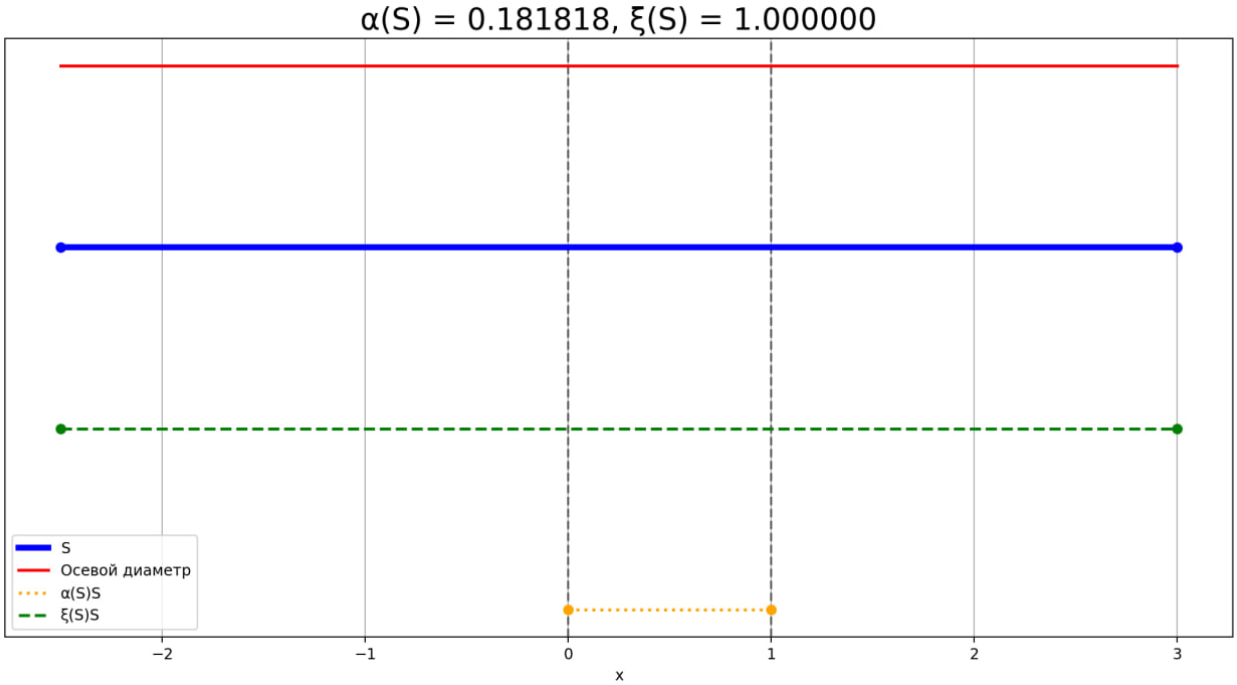

Рис. 4: S $([-2.5, 3])$ покрывает Q_1 $([0, 1])$

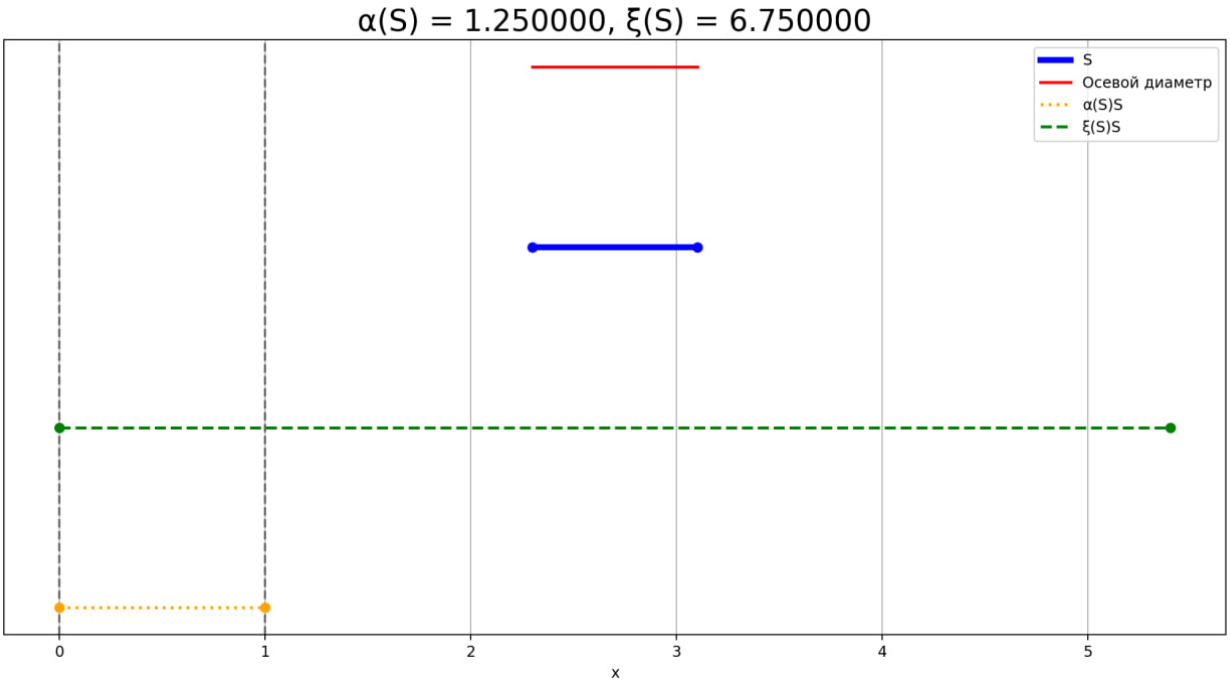

Рис. 5: S ($[2.3, 3.1]$) вне Q_1 ($[0, 1]$)

5.2 Примеры для $\mathbb{R}^2$

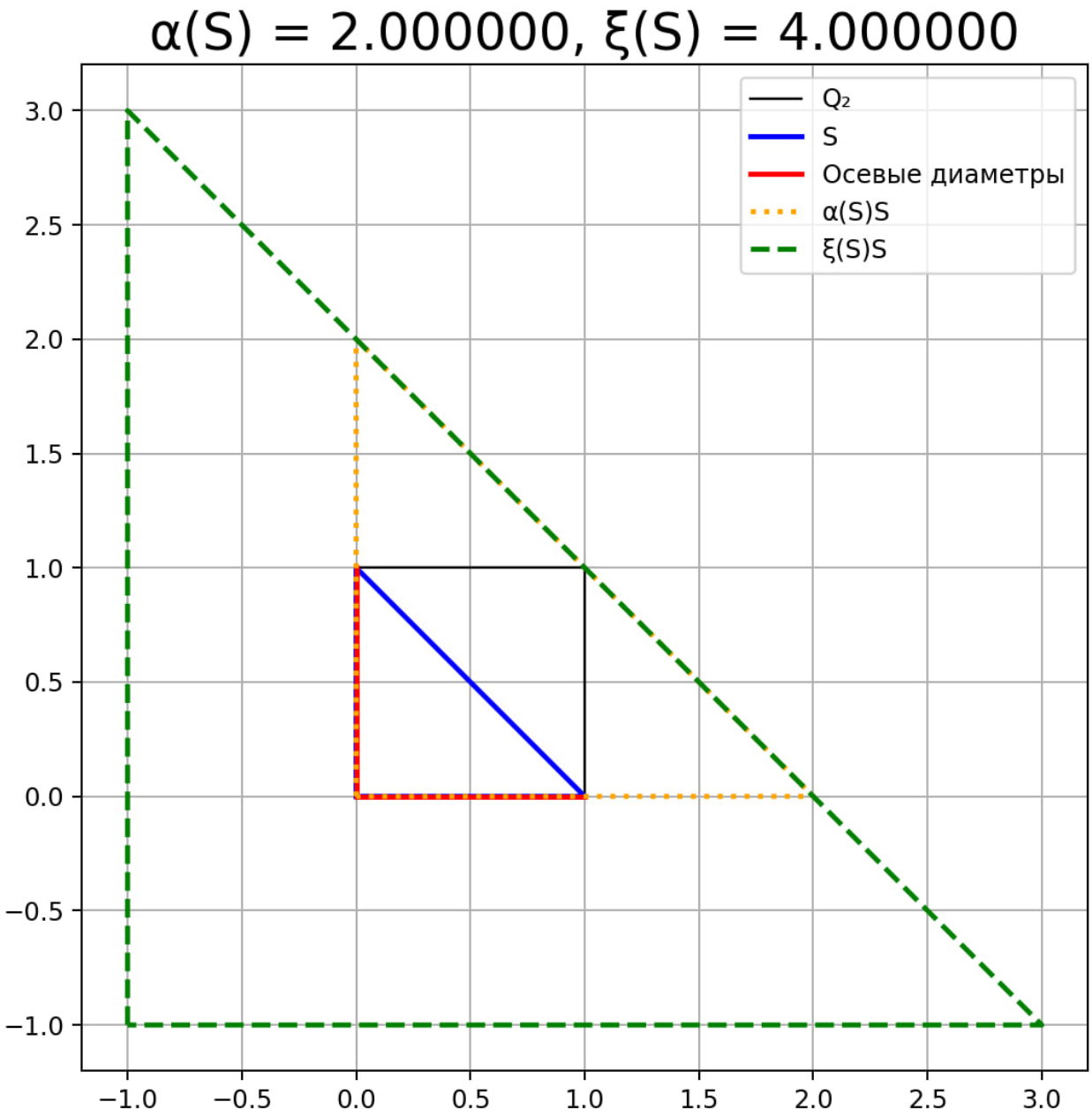

Рис. 6: S внутри Q_2 ($[0,1]^2$)

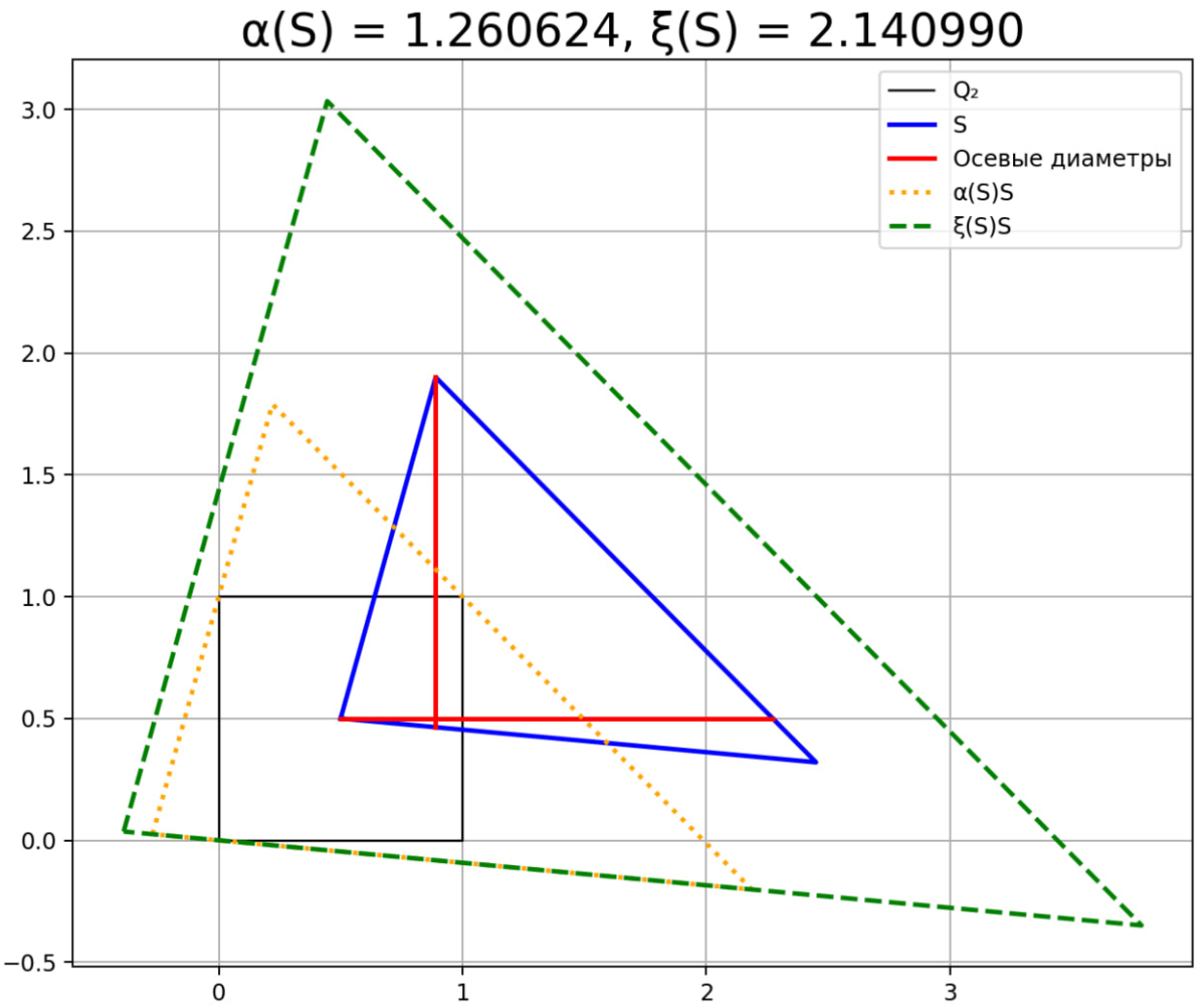

Рис. 7: S частично в Q_2 ($[0,1]^2$)

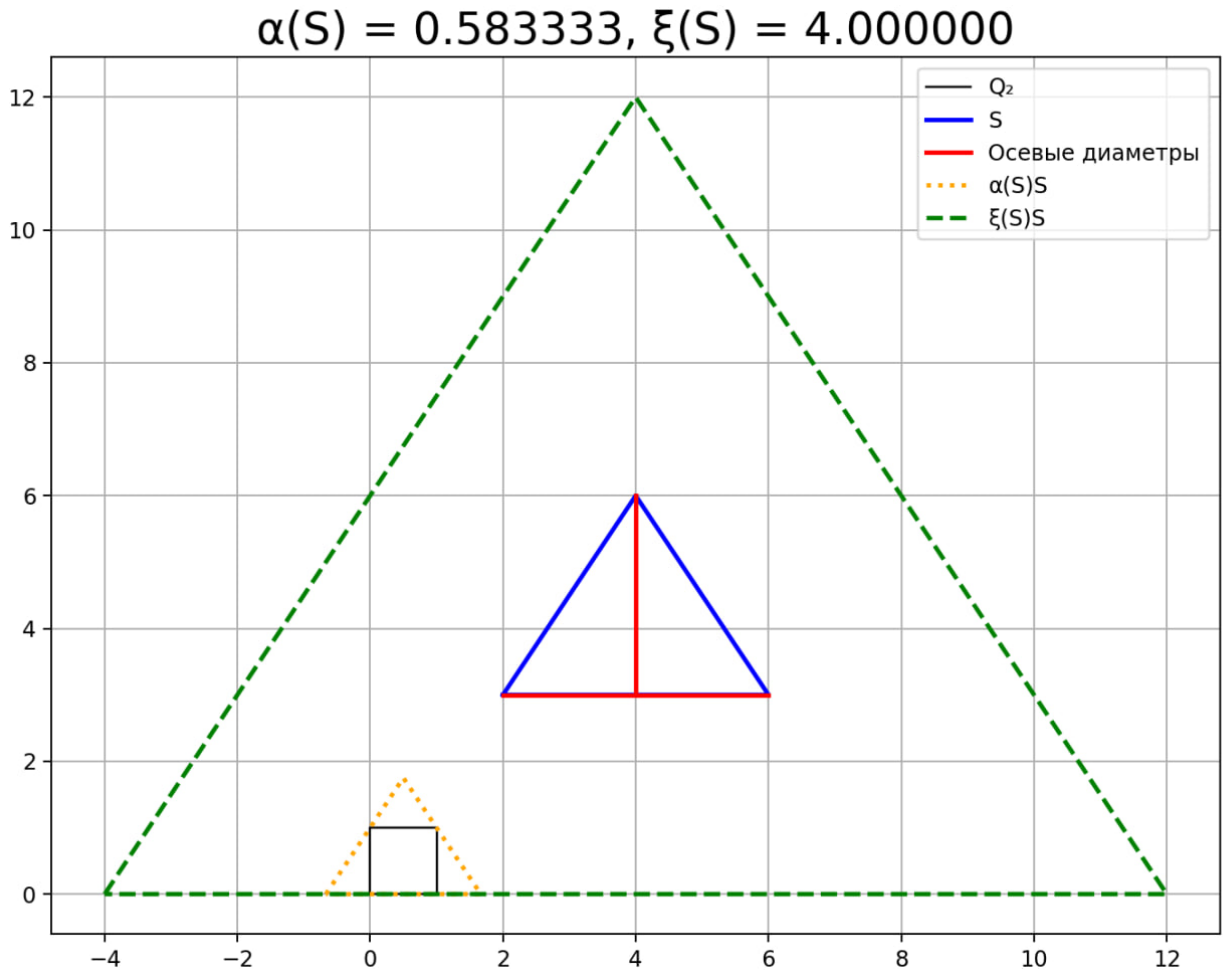

Рис. 8: S вне Q_2 ($[0,1]^2$)

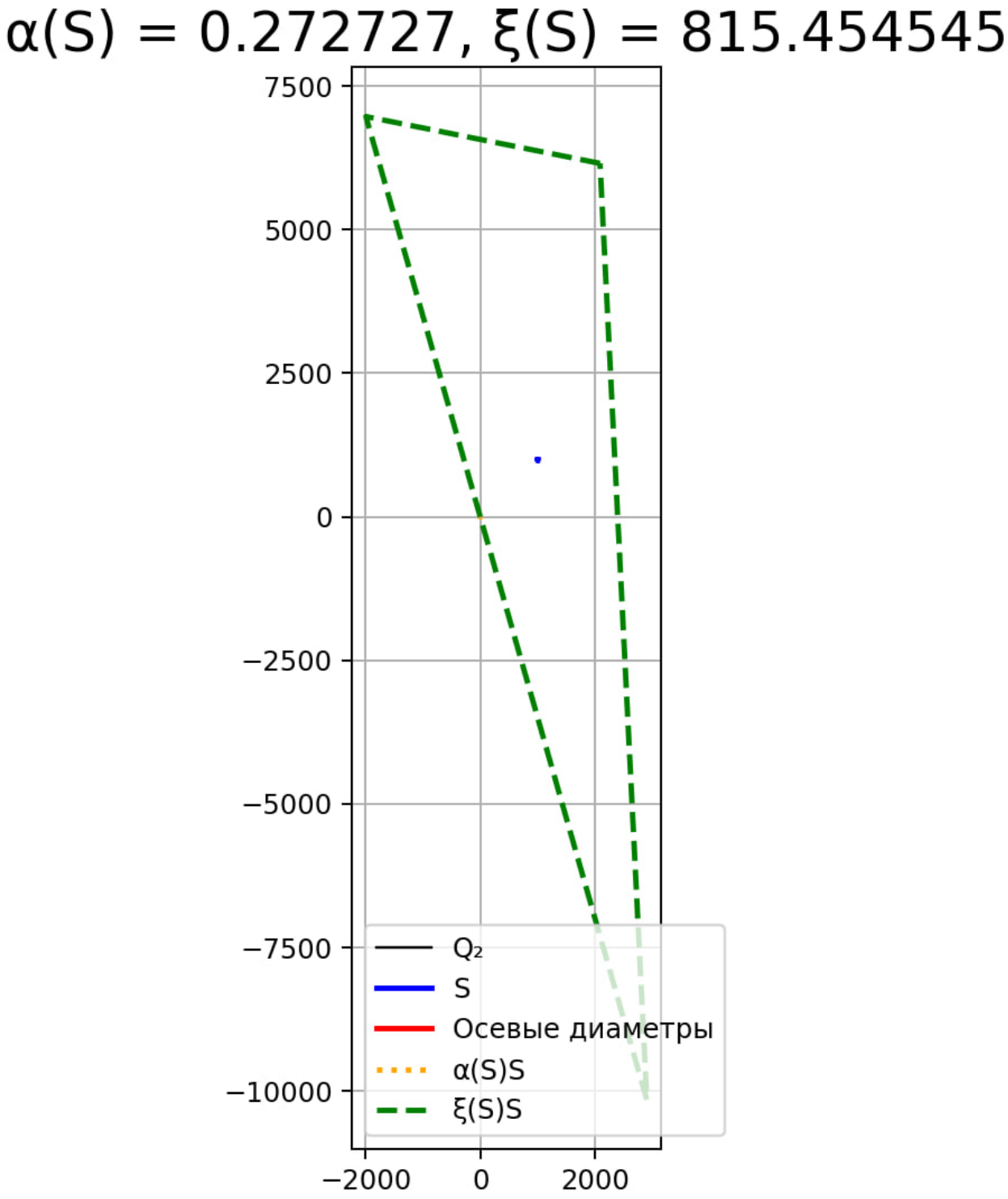

Рис. 9: S далеко вне Q_2 ($[0,1]^2$) (1)

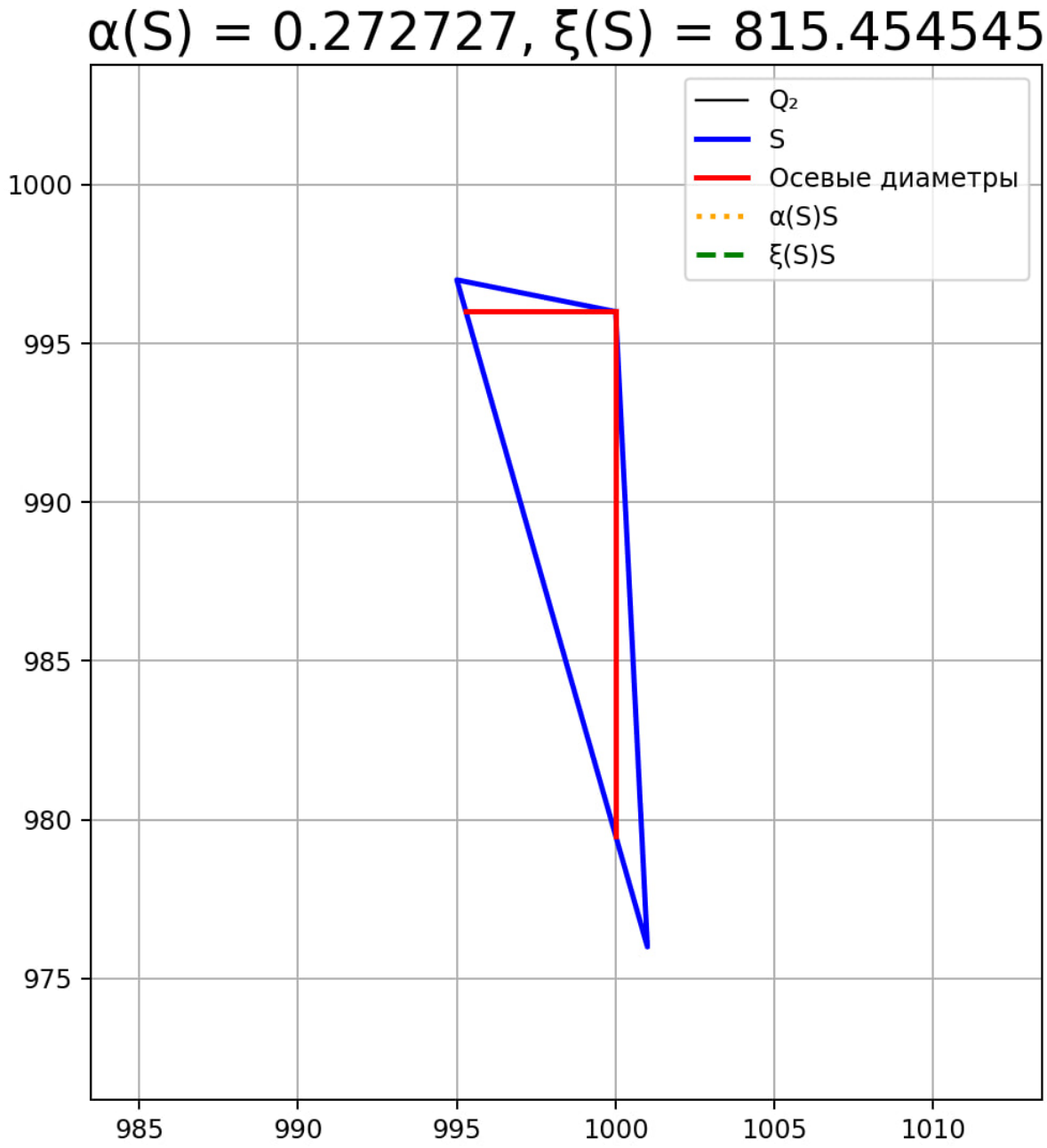

Рис. 10: S далеко вне Q_2 ($[0,1]^2$) (2)

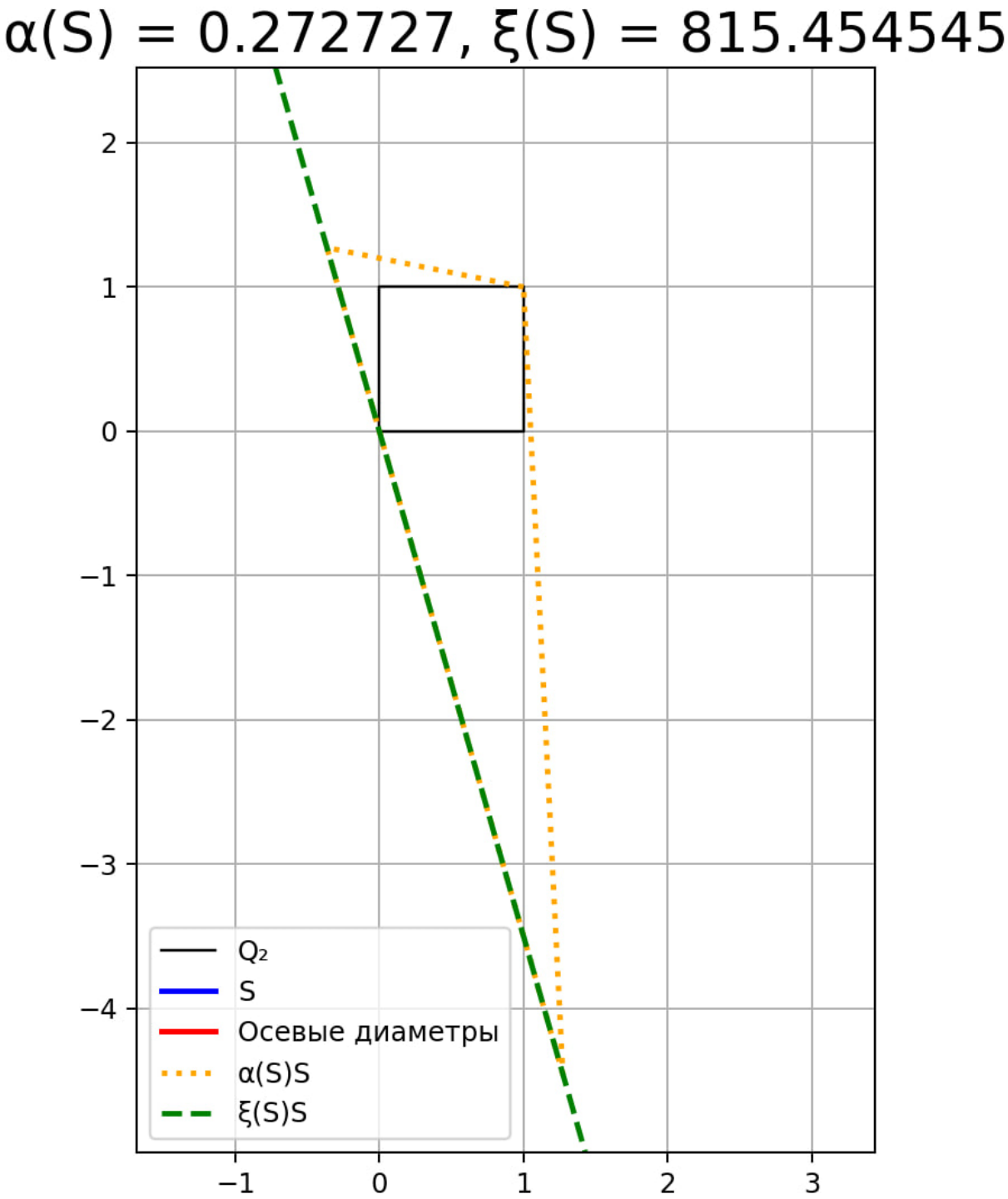

Рис. 11: S далеко вне Q_2 ($[0,1]^2$) (3)

5.3 Примеры для $\mathbb{R}^3$

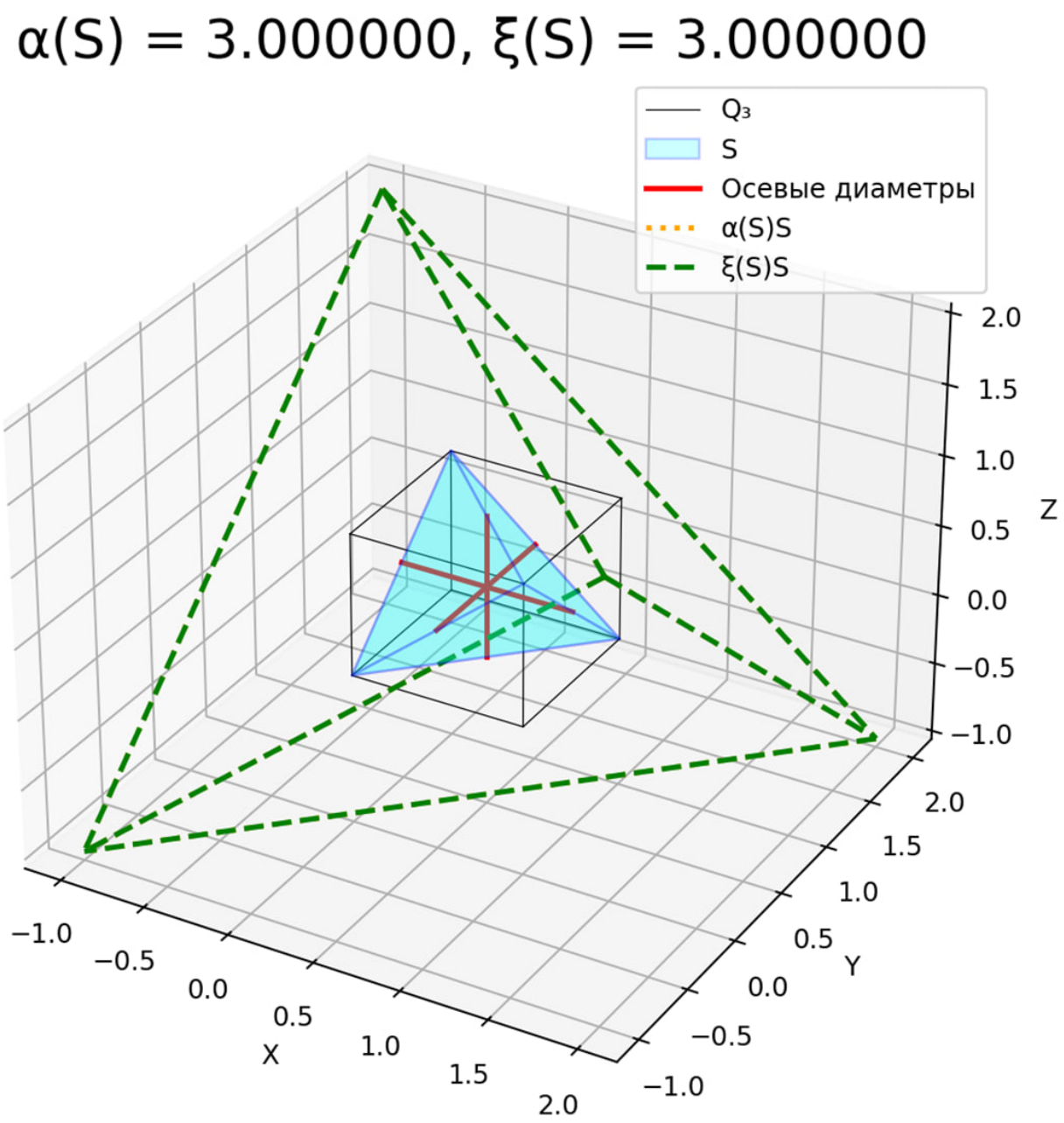

Рис. 12: S внутри Q_3 ($[0,1]^3$)

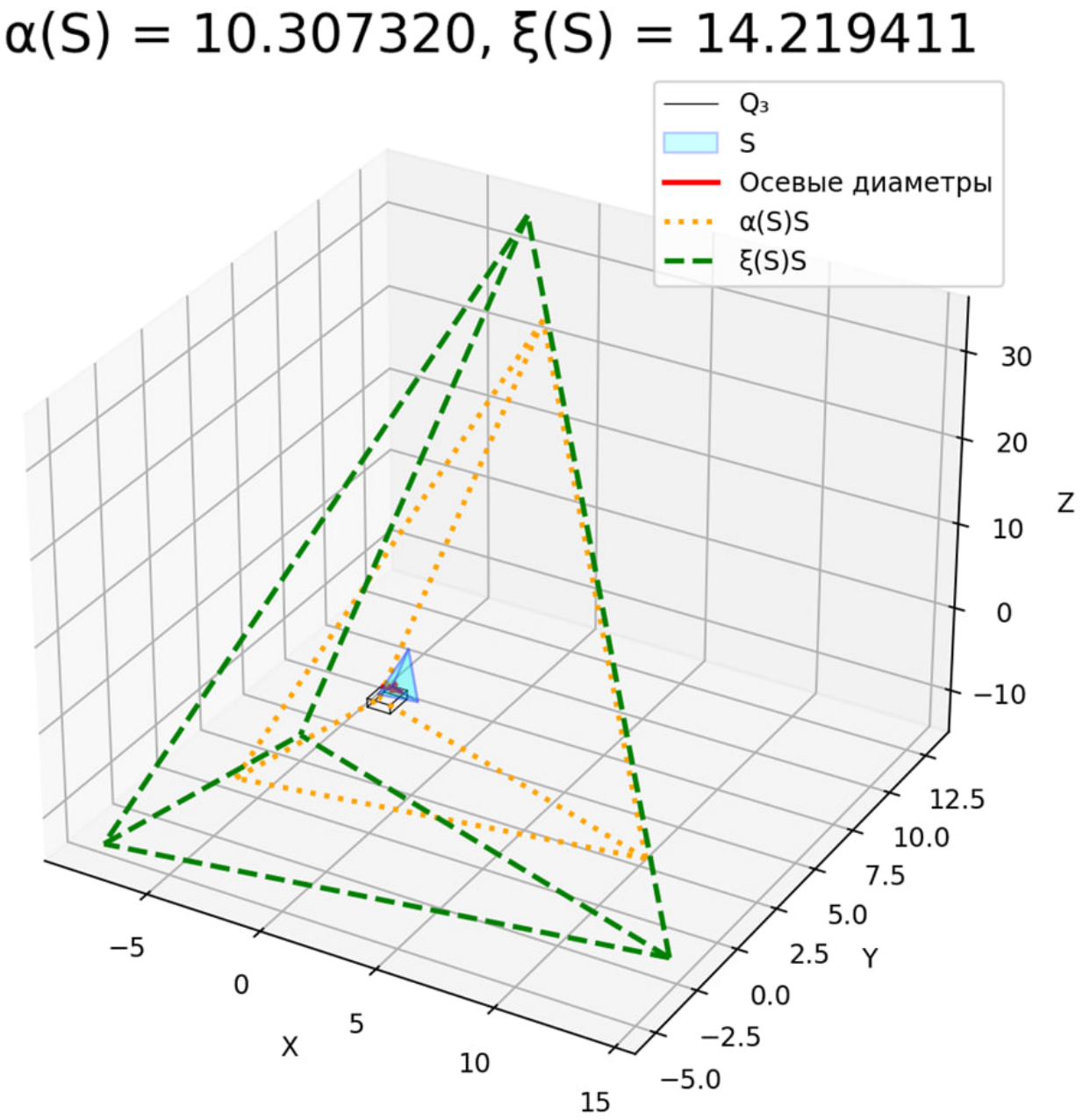

Рис. 13: S частично покрывает Q_3 ($[0,1]^3$) (1)

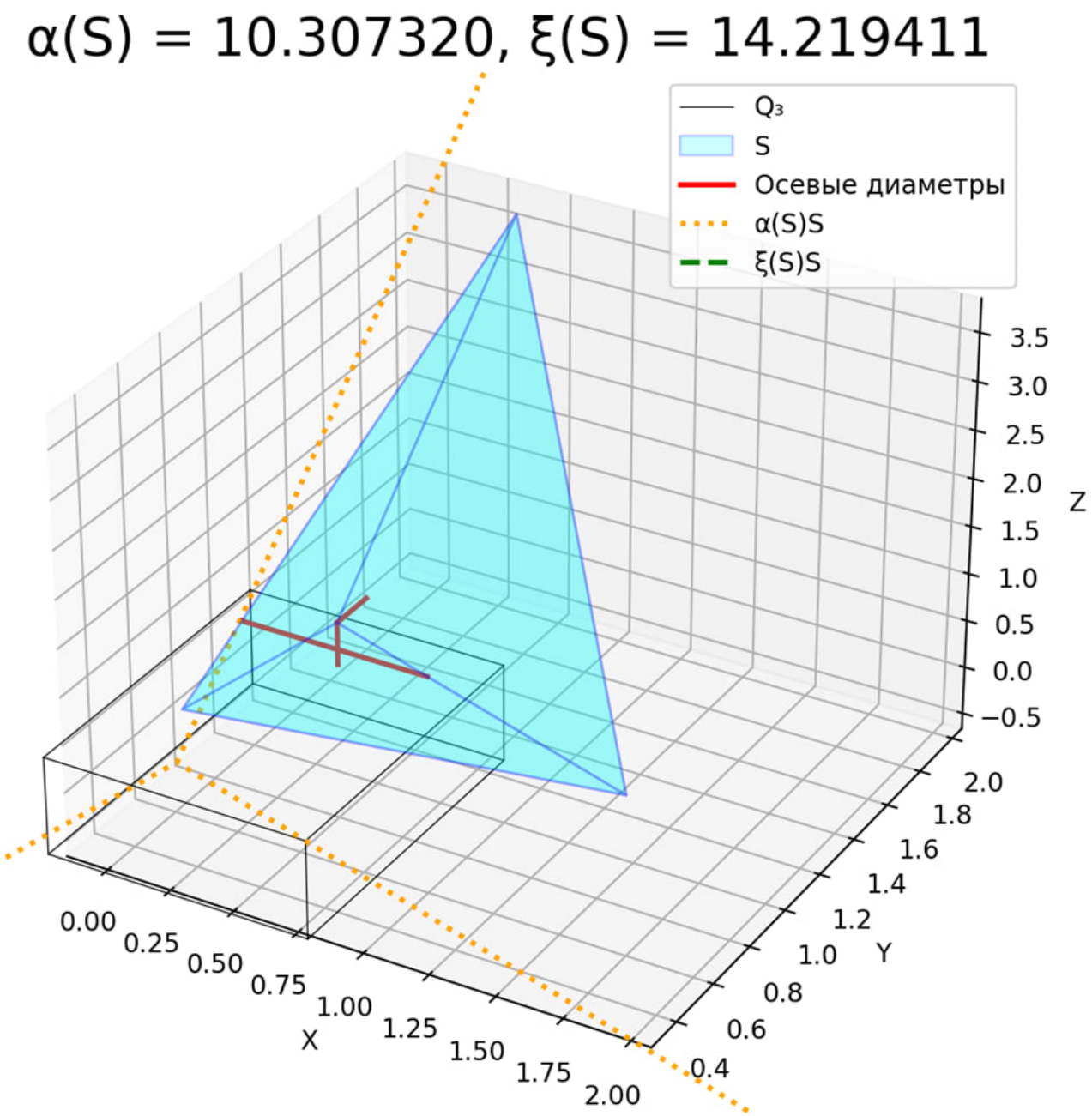

Рис. 14: S частично покрывает Q_3 ($[0,1]^3$) (2)

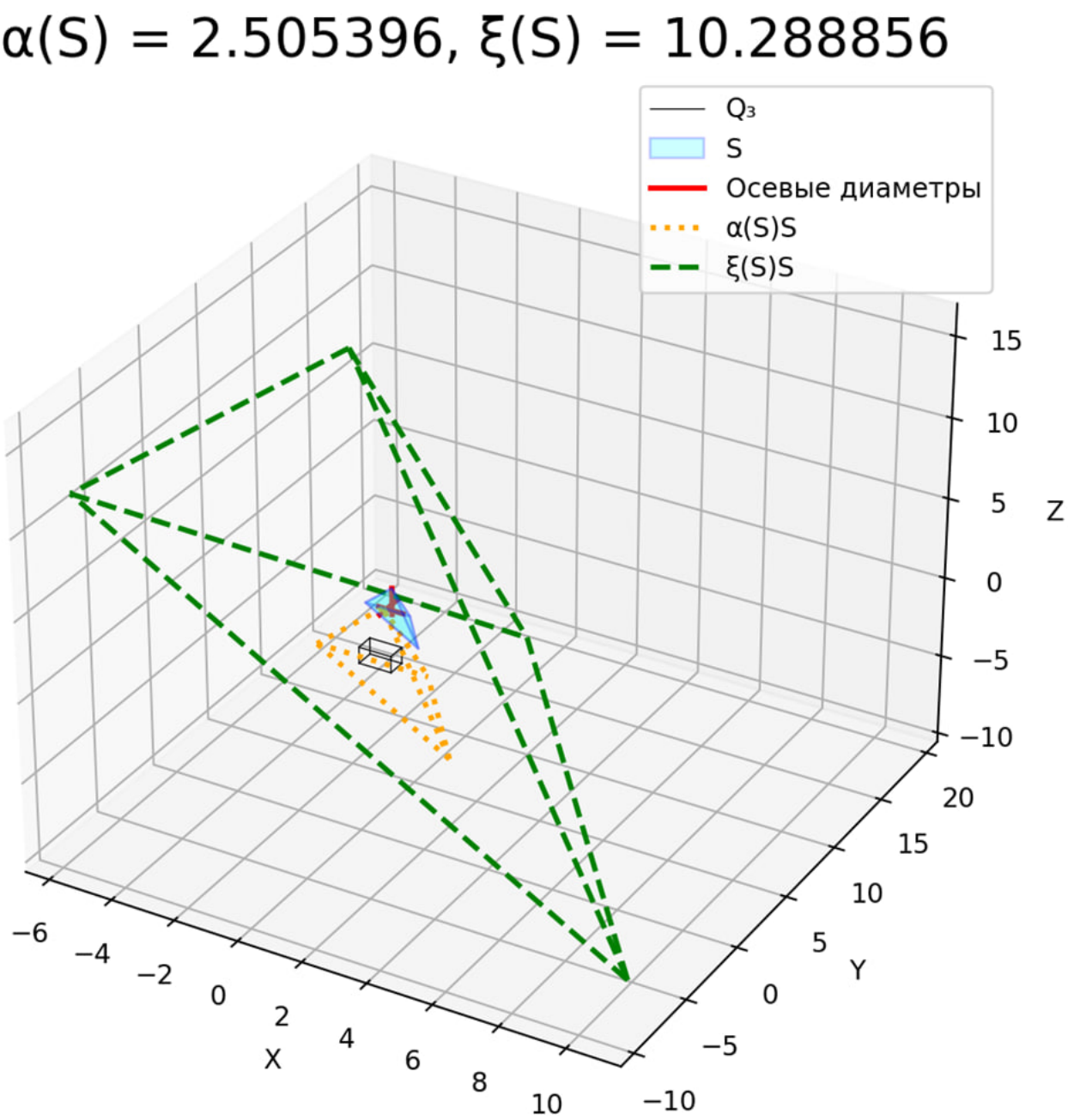

Рис. 15: S вне Q_3 ($[0,1]^3$) (1)

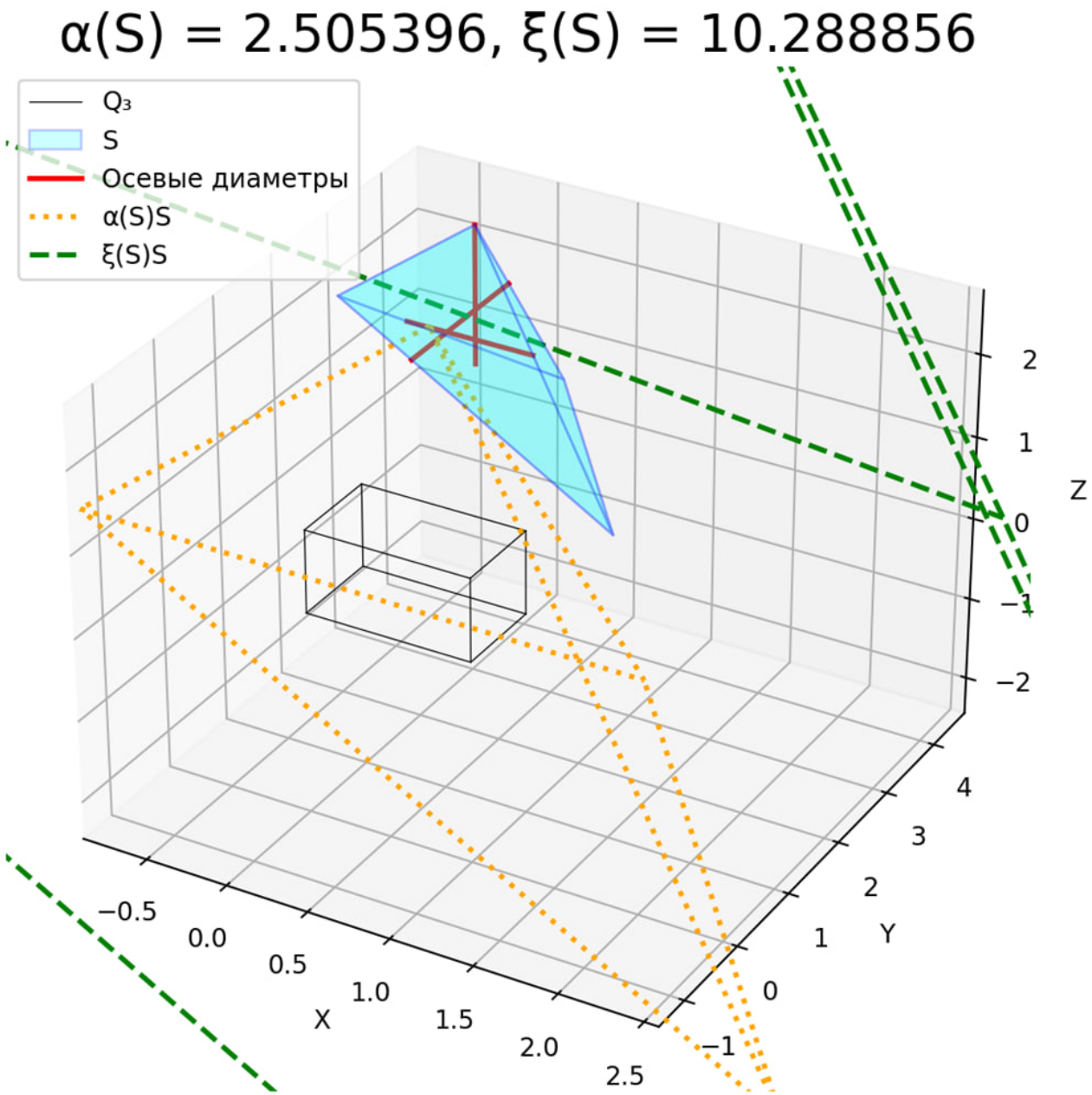

Рис. 16: S вне Q_3 ($[0,1]^3$) (2)

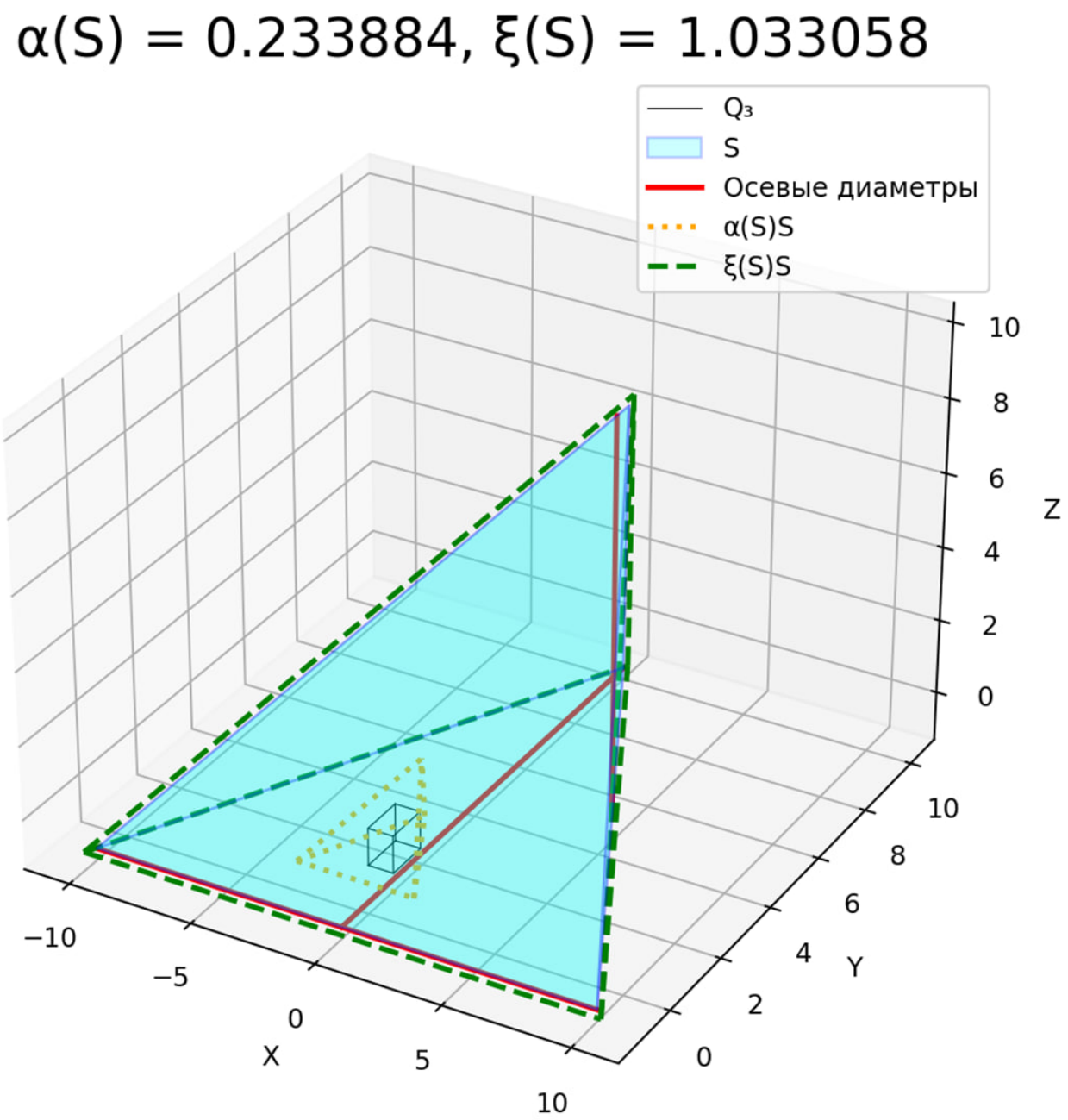

Рис. 17: S покрывает Q_3 ($[0,1]^3$)

5.4 Примеры для $\mathbb{R}^5$

```
PS D:\PycharmProjects\Laba> python main.py examples/5d.txt
*****************************************************************************************
Размерность n = 5
Вершины:
  x1 = (0.5, 1, 0.333333, 1, 1)
  x2 = (0.5, 0, 0.333333, 1, 1)
  x3 = (0.5, 0.5, 0.333333, 0, 1)
  x4 = (0.5, 0.5, 0, 0.333333, 0)
  x5 = (0, 0.5, 1, 0.333333, 0)
  x6 = (1, 0.5, 1, 0.333333, 0)
*****************************************************************************************
Строим матрицу A
A:
       0.500      1.000      0.333      1.000      1.000      1.000
       0.500      0.000      0.333      1.000      1.000      1.000
       0.500      0.500      0.333      0.000      1.000      1.000
       0.500      0.500      0.000      0.333      0.000      1.000
       0.000      0.500      1.000      0.333      0.000      1.000
       1.000      0.500      1.000      0.333      0.000      1.000
*****************************************************************************************
Ищем обратную матрицу для A
A⁻¹:
       0.000      0.000     -0.000      0.000     -1.000      1.000
       1.000     -1.000     -0.000     -0.000     -0.000     -0.000
      -0.000      0.000      0.000     -1.000      0.500      0.500
       0.500      0.500     -1.000      0.000      0.000      0.000
       0.167      0.167      0.667     -0.667     -0.167     -0.167
      -0.667      0.333      0.333      1.000      0.500     -0.500
*****************************************************************************************
```

Рис. 18: S внутри Q_5 ($[0,1]^5$) (1)

```
*****************************************************************************************
Вычисляем α(S) и ξ(S)
α(S) = 5.0
ξ(S) = 5.000002
*****************************************************************************************
Находим осевые диаметры и центр тяжести
Центр тяжести: [0.5        0.5        0.49999983 0.49999983 0.5       ]
*****************************************************************************************
ℹ Программа отработала за 0.002582 с
*****************************************************************************************
✖ Не могу визуализировать для размерности n > 3.
PS D:\PycharmProjects\Laba>
```

Рис. 19: S внутри Q_5 ($[0,1]^5$) (2)

6 Заключение

В ходе лабораторной работы создано программное обеспечение, полностью реализующее вычисление ключевых числовых характеристик симплекса – коэффициентов поглощения $\alpha(S)$ и $\xi(S)$ – на основе формул и теорем из учебного пособия М.В. Невского и А.Ю. Ухалова. Программа поддерживает произвольную размерность, а для $n \leq 3$ предоставляет наглядную визуализацию, позволяющую увидеть геометрический смысл этих величин.

Результаты тестирования на примерах из пособия полностью соответствуют ожидаемым. Программа может быть использована как в учебных целях (для изучения свойств симплексов), так и для исследовательских задач, связанных с поиском экстремальных симплексов и проверкой гипотез о значениях ξ_n.

Исходный код на Python оформлен в виде модулей, содержит подробные комментарии и доступен для дальнейшего расширения (например, поддержка других выпуклых тел или более сложных визуализаций).